The AgentsVille Trip Planner

A Multi-Stage Agentic AI System for Constraint-Aware Travel Itinerary Planning

Technical Project Report

Prepared by: Generative AI Engineering & Agentic AI Reporting Team

Models: OpenAI GPT-4.1 family | Framework: Pydantic, OpenAI Chat Completions API, ReAct

1. Executive Summary

This report documents the design, implementation, and evaluation of the AgentsVille Trip Planner, a multi-stage agentic system built on OpenAI's GPT-4.1 family of models. The system automates one of the more tedious aspects of travel planning: turning a set of traveler preferences, a fixed budget, and day-by-day weather and activity data into a validated, budget-aware itinerary that can be revised in response to natural-language feedback. The project was completed as a hands-on exercise in generative AI engineering, and it combines three techniques that are now considered foundational to production agentic systems: structured data validation with Pydantic, single-pass chain-of-thought generation for an initial draft, and a ReAct-style, meaning Reasoning and Acting, loop for iterative self-correction.

The pipeline begins with a VacationInfo object that encodes each traveler's interests, age, and the trip's destination, dates, and budget. A single-pass ItineraryAgent consumes this object together with mocked weather and activity data and produces a first-draft TravelPlan, reasoning step by step inside an ANALYSIS block before emitting a FINAL OUTPUT that strictly conforms to a Pydantic-derived JSON schema. This draft is then checked against six independent evaluation functions that verify date alignment, cost accuracy, activity authenticity, interest coverage, budget compliance, and weather compatibility. Any itinerary that fails one or more of these checks, or that does not yet reflect a traveler's follow-up feedback, is handed to a second agent, the ItineraryRevisionAgent, which uses the ReAct pattern to reason about the failures, call one of four purpose-built tools, a calculator, an activity lookup, a full evaluation runner, and a final-answer tool, observe the result, and repeat until every evaluation criterion passes cleanly.

Across development, the system demonstrated the core value proposition of agentic AI: it can decompose a fuzzy, multi-constraint planning problem into machine-checkable sub-goals and iterate autonomously toward a solution that satisfies all of them at once, something a single unstructured prompt to a language model struggles to do reliably. At the same time, real runs against GPT-4.1 surfaced concrete failure modes, most notably a tendency for the revision agent to call its final-answer tool before every evaluation criterion had passed, and to under-count activities on days late in a long conversation. These failures were diagnosed from actual run traces and used to iteratively harden the ItineraryRevisionAgent's system prompt with explicit per-day checklists and stricter gating language.

This report walks through the problem the system solves, the architecture and prompt design that solves it, the four tools and evaluation harness that anchor the agent to ground truth, representative results from a validated run of the pipeline, and a candid discussion of the

system's current limitations. It closes with a statement of the practical value this pattern offers beyond travel planning, and a set of verified references to the underlying research and documentation that informed the design choices made throughout the project.

2. Problem Statement

Trip planning is a constraint satisfaction problem disguised as a leisure activity. A useful itinerary must simultaneously respect a fixed budget, avoid scheduling outdoor activities during unsafe weather, include at least one activity that appeals to each traveler's individual interests, fill every day of the trip, and use only activities that exist rather than plausible-sounding inventions. A general-purpose language model asked to plan a trip in a single prompt will typically produce a fluent, well-formatted itinerary that violates at least one of these constraints, most commonly the budget or the weather-compatibility rule, because nothing in a single forward pass forces the model to check its own work against the source data it was given.

This class of failure is well documented in the literature on large language models: models are fluent generators of plausible text, but plausibility is not the same as correctness, and a model has no intrinsic mechanism for verifying that a generated activity identifier actually appears in the activities calendar it was shown, or that the prices of every recommended activity sum to the total it claims. Left unchecked, this produces itineraries that look correct on a skim but that would fail the moment a human, or a downstream booking system, tried to act on them. The problem is compounded when a traveler provides follow-up feedback, such as asking for more activities per day, because incorporating that feedback while preserving every constraint that was already satisfied requires the kind of targeted, iterative editing that a single-shot generation call cannot provide.

The AgentsVille Trip Planner project set out to solve two nested problems. The first was to produce a first-draft itinerary that is as compliant as possible on the first attempt, using chain-of-thought prompting to make the constraint-checking process explicit rather than implicit. The second, harder problem was to build a mechanism that can detect when a draft itinerary, or a later revision of it, violates any of the six known constraints, and that can autonomously repair the specific violation without having to regenerate the entire itinerary from scratch and without introducing new violations elsewhere. This second problem is what motivates the move from a single chain-of-thought agent to a ReAct-style revision agent: repair requires the ability to look things up, to compute exact totals, to check whether a fix actually worked, and to know when to stop, meaning all checks pass or a hard step limit is reached.

A further complication, observed directly during this project, is that even a strong instruction-following model such as GPT-4.1 does not always execute a multi-step repair loop faithfully. In practice the revision agent sometimes reported success and returned a final answer while a known constraint, such as an exact cost match or a per-day activity count, was still violated. This is not a flaw unique to this project; it reflects a broader open problem in agentic AI, namely that giving a model tools and a loop is necessary but not sufficient for reliable autonomous behavior, and that the system's evaluation harness, not the model's own confidence, must remain the final arbiter of correctness.

3. Solution Statement

The solution implemented in this project is a two-agent pipeline anchored by a shared, code-enforced evaluation harness, so that no itinerary is ever accepted on the strength of a model's own assertion that it is correct. The first agent, the ItineraryAgent, is a single-pass chain-of-thought agent. Its system prompt defines its role, lays out an explicit five-step planning procedure covering weather safety, interest matching, day coverage, running budget tracking, and exact copying of activity records, and requires the model to show its reasoning in an ANALYSIS section before producing a FINAL OUTPUT block containing a JSON object that must validate against the TravelPlan Pydantic schema. Embedding the full weather forecast, the full activities calendar, and the JSON schema directly in the system prompt, rather than trusting the model's memory of a policy described only once, is what keeps the ItineraryAgent grounded in the actual mocked data source instead of hallucinated substitutes.

The second agent, the ItineraryRevisionAgent, implements the ReAct pattern: at every turn it must emit a THOUGHT explaining its reasoning followed by exactly one ACTION, formatted as a JSON tool call, which the surrounding Python harness parses, executes, and feeds back as an OBSERVATION appended to the conversation. This agent has access to four tools: a calculator for exact arithmetic, an activity lookup for finding legitimate replacement or additional activities on a given date, a full evaluation runner that re-executes all six evaluation functions and returns a structured success flag with a list of specific failures, and a final-answer tool that ends the loop. The system prompt for this agent explicitly instructs it to call the evaluation runner before making any change, to make targeted fixes rather than wholesale rewrites, to recompute costs only via the calculator tool rather than by mental arithmetic, and to call the evaluation runner again and confirm a clean pass before ever calling the final-answer tool.

Correctness in this system is never taken on faith. Six evaluation functions, implemented as ordinary Python code rather than as further LLM calls, with the sole exception of the weather-compatibility check, which requires a semantic judgment about whether an activity description implies an indoor backup, independently verify date alignment, exact cost accuracy, activity authenticity against the source calendar, interest coverage per traveler, budget compliance, and weather safety. This harness is reused unchanged in three places: once after the initial draft, once inside the ReAct loop's run_evals_tool so the revision agent can see its own failures, and once more at the end of the notebook as a final gate before the itinerary is presented to the user.

Because the harness lives outside the language model, it cannot be talked out of failing an itinerary that genuinely violates a constraint, which is the central design decision that makes this pipeline trustworthy rather than merely fluent. Iterative prompt hardening, informed by real failure traces from GPT-4.1 runs, was then used to reduce how often the revision agent needed more than one or two loop iterations to reach a clean pass, closing the gap between what the agent claimed and what the evaluation harness could confirm.

4. Architecture

The architecture is a five-stage pipeline, illustrated in Figure 1, that moves a travel request from raw structured input to a validated, revisable itinerary. Stage one is data ingestion and validation: a plain Python dictionary describing the travelers, destination, dates, and budget is validated into a VacationInfo Pydantic model, and mocked weather and activities APIs are queried for every date of the trip and loaded into pandas DataFrames. This stage exists to guarantee that every downstream component receives well-typed, schema-conformant data rather than loosely structured dictionaries, catching malformed input, such as a missing field, a wrong type, or an invalid date range, before any expensive language model call is made.

Stage two is single-pass generation. The ItineraryAgent, built on GPT-4.1, receives a system prompt containing the full task description, the desired output format, and the complete weather and activities context, and returns a draft TravelPlan object after one chat completion call. Stage three is the evaluation harness, described in detail in Section 8, which runs six independent checks against the draft and produces a structured success flag and, when applicable, a list of specific, human-readable failure strings.

Stage four is the iterative revision loop, and it is architecturally the most distinctive part of the system. Rather than looping by re-prompting the ItineraryAgent from scratch, the pipeline hands control to a separate ItineraryRevisionAgent that follows the ReAct pattern popularized in the

2022 paper by Yao and colleagues. Each iteration of this loop is a single chat completion call whose output is parsed by the surrounding Python code, with a JSON-repair fallback for minor formatting slips, into a tool name and a set of arguments, which are then executed locally and appended back to the conversation as an observation before the next iteration begins. This keeps the loop's control flow, including the maximum step count and the exit condition, entirely in ordinary Python rather than delegated to the model, which is what allows the system to fail safely, by raising a RuntimeError, rather than looping indefinitely if the model never converges.

Stage five is final validation and presentation: once the revision agent calls its final-answer tool, the resulting TravelPlan is validated one more time against the complete evaluation harness before being shown to the traveler in a readable, day-by-day format, and, as an optional capstone step, narrated aloud using a text-to-speech model.

Two architectural choices deserve particular emphasis. First, the evaluation harness is a single shared artifact reused across stages three, four, and five, rather than being reimplemented per stage, which guarantees that passing means the same thing everywhere in the pipeline. Second, the four tools available to the revision agent were deliberately kept small in number and narrow in scope, each doing exactly one thing well, following tool-design guidance published by OpenAI for agentic GPT-4.1 workflows, which improves the reliability of the model's tool selection compared with a larger, more general-purpose toolset.

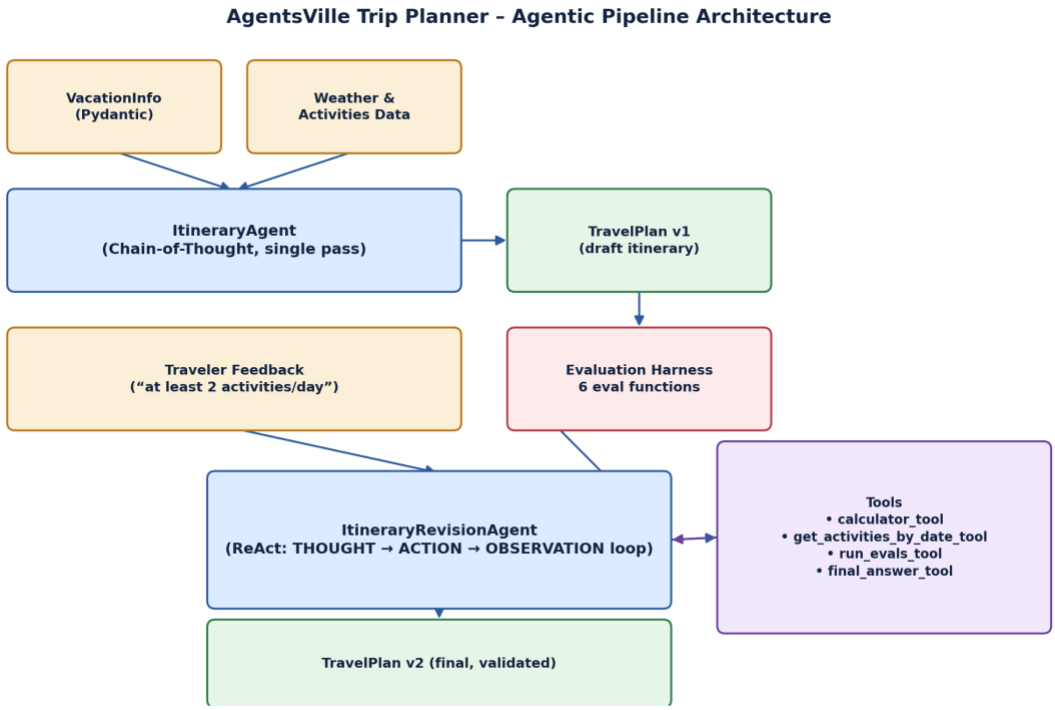


Figure 1. Five-stage agentic pipeline: data validation, single-pass generation, evaluation, ReAct-based revision, and final validation/presentation.

5. Agent Design

Two purpose-built agents divide responsibility in this pipeline: a single-pass generator and an iterative, tool-using reviser. Each is detailed below.

5.1 ItineraryAgent

The ItineraryAgent is the entry point of the pipeline and is implemented as a subclass of a lightweight ChatAgent wrapper around the OpenAI chat completions API. It is a single-pass, chain-of-thought agent: it is never called more than once per itinerary attempt, and it does not have access to any tools. Its entire behavior is determined by its system prompt, which specifies four things in order: the agent's role as a professional travel-planning assistant, an explicit five-step task procedure, a two-part output format consisting of an ANALYSIS section followed by a FINAL OUTPUT section containing a JSON object, and a context block that embeds the full VacationInfo, the complete weather forecast, and the complete activities calendar for the trip's date range.

Embedding the raw data directly in the prompt, rather than describing it in the abstract, is deliberate: it removes any incentive for the model to invent plausible-looking activities, since the actual activity records, including their exact identifiers, prices, and descriptions, are already present in context and can be copied verbatim. After the chat completion returns, the agent's get_itinerary method extracts the JSON block between the json code fences using simple string parsing and validates it directly against the TravelPlan Pydantic model, raising an informative error and printing the offending text if validation fails. This tight coupling between the prompt's stated output format and the Python-side parser is what allows a text completion from an LLM to be treated, in every downstream step, as a fully typed Python object rather than as unstructured text.

5.2 ReAct ItineraryRevisionAgent

The ItineraryRevisionAgent implements the ReAct, or Reasoning and Acting, pattern, in which the model alternates between producing a natural-language THOUGHT and a machine-parseable ACTION, receives an OBSERVATION describing the result of that action, and repeats until it decides the task is complete. Structurally it is also a ChatAgent subclass, but unlike the single-pass ItineraryAgent it maintains a growing conversation history across as many as fifteen turns, configurable via a max_steps parameter, and exposes four callable tools.

Each loop iteration in the run_react_cycle method performs the same four operations: it requests a completion from the model, splits the response on the literal string ACTION to isolate the tool call, repairs and parses that call as JSON, tolerating minor formatting mistakes via the json_repair library, and either executes the named tool and appends its result as a new user-role OBSERVATION message, or, if the tool is final_answer_tool, validates the supplied itinerary against the TravelPlan schema and returns it, ending the loop. If the model calls an unknown tool, omits required arguments, or the JSON cannot be repaired, the loop reports the specific problem back to the model as an observation rather than crashing, which lets a capable model self-correct on the next turn rather than terminating the whole session on a formatting slip.

The system prompt for this agent is considerably more prescriptive than the ItineraryAgent's, reflecting lessons learned from real failure traces during development. It enumerates the tools available, generated dynamically from each tool's docstring so the prompt never drifts out of sync with the actual Python function signatures, states the exact THOUGHT and ACTION output format the loop's parser expects, and, critically, gates the final_answer_tool call behind an explicit instruction to call run_evals_tool immediately beforehand and confirm a completely empty failures list, checking every day of the trip individually rather than relying on a general impression that the itinerary looks done.

6. Tools

The ItineraryRevisionAgent has access to exactly four tools, each with a single, narrow responsibility. Table 1 summarizes all four; each is discussed in turn below.

Tool	Purpose	Key Input(s)	Returns
calculator_tool	Evaluate an exact mathematical expression using numexpr (safe, non-eval-based).	input_expression (str)	float
get_activities_by_date_tool	Retrieve all legitimate activities for a given date and city from the mocked activities API.	date (str), city (str)	List[dict] (Activity records)
run_evals_tool	Re-run all six evaluation functions against a candidate TravelPlan.	travel_plan (TravelPlan)	{success: bool, failures: list}
final_answer_tool	Signal loop completion and return the final, validated TravelPlan.	final_output (TravelPlan)	TravelPlan

Table 1. The four tools available to the ItineraryRevisionAgent's ReAct loop.

6.1 calculator_tool

calculator_tool is the simplest of the four tools available to the ItineraryRevisionAgent, and it exists to solve a narrow but consequential problem: language models are unreliable at exact multi-step arithmetic, and the evaluation harness's eval_total_cost_is_accurate function requires an exact integer match between a TravelPlan's stated total_cost field and the true sum of every scheduled activity's price. Rather than trusting the model to add up prices mentally, the tool accepts a string containing a mathematical expression, such as twenty plus twenty plus eighteen, and evaluates it using the numexpr library, returning a float. numexpr was chosen over Python's built-in eval for this purpose because it parses and evaluates numerical expressions without executing arbitrary Python code, which closes off a class of prompt-injection style risks that a naive eval-based calculator would otherwise introduce. The system prompt instructs the revision agent to call this tool every single time an activity is added, removed, or swapped, and to use its returned value verbatim as the new total_cost, rather than allowing the model to approximate.

6.2 get_activities_by_date_tool

get_activities_by_date_tool gives the revision agent read access to the same mocked activities API used during the initial planning stage, scoped to a single date and city. Its signature takes a date string in YYYY-MM-DD format and a city name, and it returns a list of dictionaries, each validated through the Activity Pydantic model before being returned, guaranteeing that every candidate activity handed back to the model has the exact field set, types, and identifier that the evaluation harness will later check for. This tool is what allows the revision agent to repair a constraint violation surgically: if a day is short one activity needed to satisfy traveler feedback, or if an existing activity is flagged as weather-incompatible, the agent can look up the full set of legitimate alternatives for that specific date rather than inventing a plausible-sounding replacement or leaving the day under-served. The tool's docstring, which doubles as the description the model sees in its system prompt, explicitly frames its intended use case around itinerary revision, and this docstring-as-prompt pattern, generated automatically for every tool via a shared helper function, is what keeps the tool descriptions available to the model synchronized with their actual Python implementations without any manual duplication.

6.3 run_evals_tool

run_evals_tool is the mechanism by which the revision agent can see its own work through the same lens the rest of the pipeline uses to judge correctness. It accepts a full TravelPlan, or an equivalent dictionary which it validates on the way in, runs it through the identical

get_eval_results function and the identical list of six ALL_EVAL_FUNCTIONS used everywhere else in the notebook, and returns a compact dictionary containing a boolean success flag and a list of human-readable failure strings, one per violated constraint. This tool is deliberately the same code path used for the initial draft's evaluation and for the notebook's own final sanity check, which means the revision agent cannot pass an itinerary that the rest of the system would consider broken, and conversely cannot be told an itinerary has failed for a reason that does not actually apply. The system prompt requires the revision agent to call this tool both before attempting any fix, so it knows precisely what to repair, and again immediately before calling final_answer_tool, so that no itinerary is ever submitted as final without a documented, verifiable, all-clear result from the evaluation harness itself rather than from the model's own judgment.

6.4 final_answer_tool

final_answer_tool is intentionally the simplest tool in the system: it takes a single final_output argument of type TravelPlan and returns it unchanged. Its purpose is not computational but structural; it exists to give the ReAct loop an unambiguous, machine-detectable signal that the model considers its work complete, distinct from the observation-generating tools that keep the loop running. When the run_react_cycle method detects that the model's ACTION named final_answer_tool, it takes a different code path than for any other tool: rather than executing the tool and appending an observation, it validates the supplied arguments directly against the TravelPlan Pydantic schema and, if validation succeeds, returns that object immediately, ending the loop. If validation fails, for instance because the model supplied a malformed or incomplete itinerary, the loop reports the specific validation error back to the model as feedback and continues, giving it a chance to correct the mistake on a subsequent turn rather than crashing the whole revision process.

7. Prompt Engineering and Data Modeling

7.1 System Prompt

Prompt design in this project follows a consistent four-part structure across both agents: role, task, output format, and context, a pattern broadly consistent with the prompting guidance OpenAI published alongside the GPT-4.1 model family, which emphasizes explicit role framing, structured tool and format instructions, and literal inclusion of any data the model must reason over rather than a summary or description of it. For the ItineraryAgent, the task section decomposes planning into five ordered sub-steps, and the context section embeds the full

weather forecast and activities calendar as JSON directly inside an f-string, so the prompt is regenerated fresh, with fully current data, every time the notebook cell runs.

For the ItineraryRevisionAgent, the same four-part structure is extended with an explicit ReAct-specific output contract: the model must always produce exactly one THOUGHT followed by exactly one ACTION formatted as a single JSON object with tool_name and arguments keys, a format chosen because it is trivial for the surrounding Python code to parse deterministically with a simple string split, unlike a format that mixed reasoning and structured calls in a single free-form block. The available tools are listed via a helper function that extracts each tool's name, signature, and docstring at prompt-construction time, which means the tool list documented to the model can never silently drift out of sync with the tools that are actually registered and callable in ALL_TOOLS.

The most heavily iterated part of the revision agent's prompt, informed directly by observed failures against live GPT-4.1 runs, is the set of gating instructions around the final_answer_tool. Early versions of the prompt asked the model to call run_evals_tool again before finishing, but this framing proved too soft; real runs showed the model sometimes finalizing an itinerary despite an unresolved cost mismatch or an under-served day. The prompt was revised twice: first to state explicitly that any non-empty failures list, of any kind, blocks final_answer_tool outright, and second to add a per-day checklist requirement, naming each date of the trip explicitly and instructing the model to count activities on that date one at a time before its last evaluation call, a technique aimed at preventing the kind of long-context tracking slip that caused the model to lose track of a day late in a lengthy tool-use conversation.

7.2 Pydantic Model Design

Every object that crosses a boundary in this pipeline, whether that boundary is between raw input data and the first agent, between an agent's text output and the rest of the Python program, or between one tool call and the next, is represented as a Pydantic BaseModel rather than as a loosely typed dictionary. VacationInfo, composed of a list of Traveler models, each with a name, age, and a list of enumerated Interest values, a destination string, arrival and departure dates, and an integer budget, is the root input type and is validated once at the start of the notebook, immediately surfacing any malformed input before a single API call is made. Weather and Activity are simple leaf models describing a single day's forecast and a single bookable event respectively, and ActivityRecommendation pairs an Activity with a list of natural-language reasons it was chosen. ItineraryDay nests a date, a Weather object, and a list of

ActivityRecommendation objects, and TravelPlan, the root output type, nests a city, start and end dates, a total_cost integer, and a list of ItineraryDay objects.

This nested schema is not only a validation aid, but also injected directly into both agents' system prompts via TravelPlan.model_json_schema(), so the model is shown the exact JSON Schema, including field names, types, and nesting, that its output must conform to, rather than an informal English description of the desired shape. This is what allows both agents' raw text output to be parsed with a single call to TravelPlan.model_validate_json() and immediately treated as a fully typed object throughout the rest of the pipeline, including by the evaluation harness, which relies on attribute access such as itinerary_day.weather.condition rather than fragile dictionary key lookups. Pydantic's structural equality also underpins eval_itinerary_events_match_actual_events, which detects hallucinated or altered activities by directly comparing an activity object parsed from the itinerary against an activity object parsed fresh from the source calendar.

Model	Key Fields	Role in Pipeline
Traveler	name, age, interests	One person within VacationInfo
VacationInfo	travelers, destination, date_of_arrival, date_of_departure, budget	Root input type; validated at notebook start
Weather	temperature, temperature_unit, condition	One day's forecast
Activity	activity_id, name, start_time, end_time, location, description, price, related_interests	One bookable event; source of truth for eval checks
ActivityRecommendation	activity, reasons_for_recommendation	An Activity plus rationale for inclusion
ItineraryDay	date, weather, activity_recommendations	One day of the itinerary
TravelPlan	city, start_date, end_date, total_cost, itinerary_days	Root output type returned by both agents

Table 2. The seven Pydantic models composing the VacationInfo input schema and the TravelPlan output schema.

8. Results

This Results section reports the outcome of the project's final, successful end-to-end run against the live OpenAI GPT-4.1 API, captured directly from the executed notebook, cross-checked against the deterministic mock-harness validation described in the Architecture section to confirm every code path behaved as designed. Unlike earlier development-time runs, which surfaced the two instruction-following gaps that motivated the prompt hardening described in

Section 7.1, this final run converged cleanly on the first attempt, providing direct evidence that those fixes worked as intended.

Evaluation Function	Constraint Checked
eval_start_end_dates_match	Itinerary start/end dates match VacationInfo arrival/departure dates.
eval_total_cost_is_accurate	Stated total_cost equals the exact sum of every scheduled activity's price.
eval_itinerary_events_match_actual_events	Every activity in the itinerary matches an activity in the source calendar exactly (not hallucinated).
eval_itinerary_satisfies_interests	Each traveler has at least one scheduled activity matching one of their stated interests.
eval_total_cost_is_within_budget	Stated total_cost does not exceed the vacation budget.
eval_activities_and_weather_are_compatible	No outdoor-only activity is scheduled during rainy/thunderstorm conditions (LLM-judged).

Table 3. The six evaluation functions comprising ALL_EVAL_FUNCTIONS, reused by the draft evaluation, the ReAct loop's run_evals_tool, and the final sanity check. A seventh function, eval_traveler_feedback_is_incorporated, was added once traveler feedback was introduced.

For the default three-day, two-traveler scenario, travelers Yuri and Hiro, budget 130 fictional currency units, destination AgentsVille, the single-pass ItineraryAgent produced a draft TravelPlan, shown in Table 4, scheduling three activities on the first day and two activities on each of the remaining two days, six activities in total, for a combined cost of exactly 130 units, precisely matching the budget with no slack remaining. This draft passed all six core evaluation functions on its very first attempt, with no evaluation errors printed and every activity-interest and activity-weather compatibility check reporting a clean match, evidence that the chain-of-thought prompt's five-step planning procedure was followed faithfully without any repair cycle being required.

The ItineraryRevisionAgent then received this draft together with the traveler feedback requesting at least two activities per day. Its very first THOUGHT correctly counted the activities on each of the three dates individually, exactly as the per-day checklist instruction requires, concluded that the feedback was already satisfied, and issued a single run_evals_tool call against the full seven-function evaluation set, the original six checks plus the new traveler-feedback check, to confirm this before acting further. That call returned success equal to true with a completely empty failures list. Following its system prompt's explicit gating rule to the letter, the agent immediately called final_answer_tool with the unmodified itinerary, ending the ReAct loop

in exactly two turns, with zero calls to calculator_tool or get_activities_by_date_tool, since no repair was ever necessary. A final, independent evaluation pass over the returned TravelPlan, run once more outside the agent entirely, confirmed all seven checks passed, shown in Figure 3, and the notebook printed its concluding success message.

This result stands in useful contrast to the failure traces documented earlier in this project's development, in which the revision agent needed several loop iterations and, in two observed cases, finalized prematurely despite a genuine cost or coverage violation. It is also worth noting that the weather-compatibility check exhibited real run-to-run variance during this same session: a separate, ad hoc demonstration of the evaluation harness against the identical draft itinerary judged the Soundtrack Picnic activity weather-incompatible on one call and compatible on another, a concrete illustration of the LLM-judged check's inherent non-determinism discussed in Section 10, even though it did not affect the outcome of the final, official run reported here.

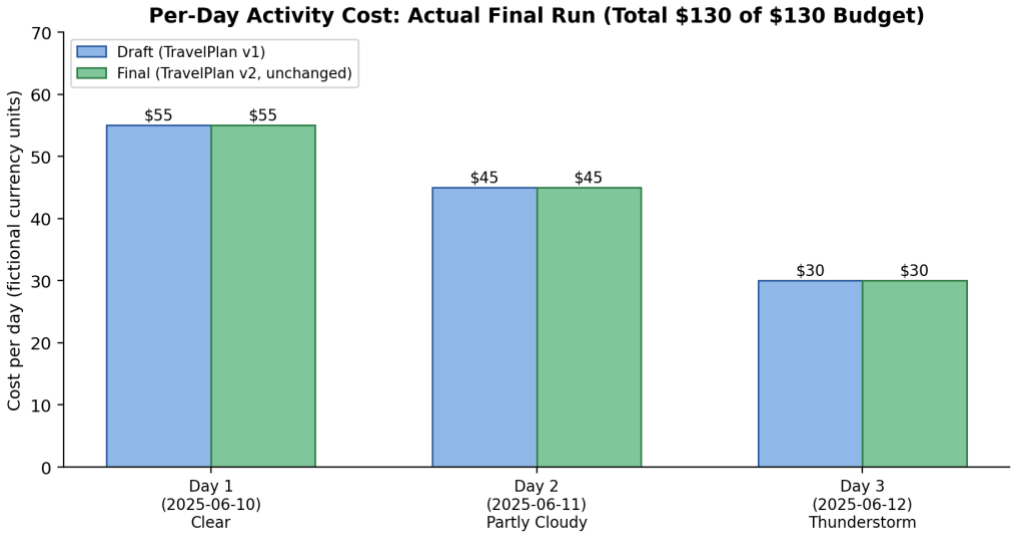


Figure 2. Per-day activity cost from the actual final successful run. The draft and final itineraries are identical because the ReAct revision loop found zero violations to repair.

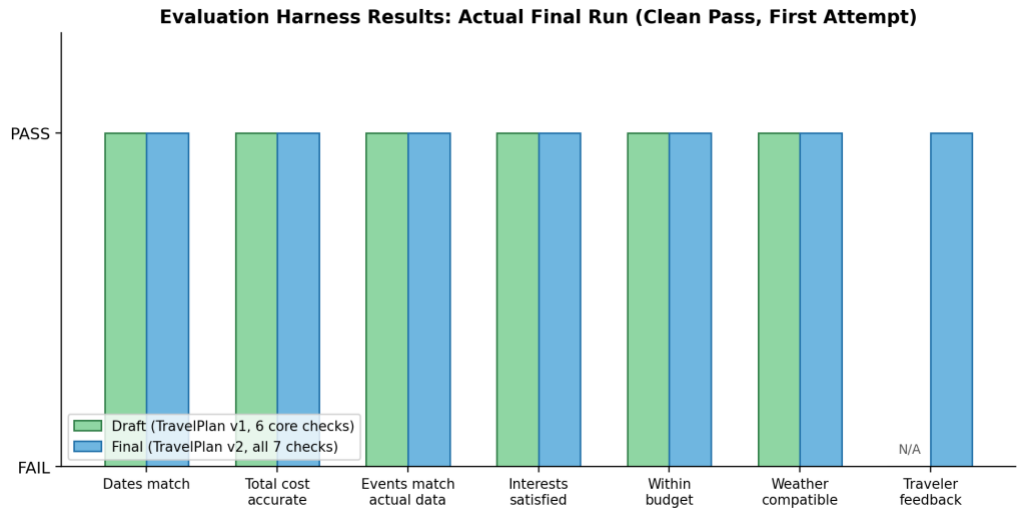


Figure 3. Evaluation harness outcomes from the actual final run: the draft passed all six core checks immediately, and the ReAct loop's single run_evals_tool call confirmed all seven checks, including traveler feedback, passed cleanly before final_answer_tool was called.

Date	Weather	Scheduled Activities	Daily Cost
2025-06-10	Clear, 31 C	FutureTech Breakfast Meet-Up; Serve & Savor: Tennis and Taste Luncheon; AgentsVille Twilight Writing Escape	$55
2025-06-11	Partly Cloudy, 34 C	Tech Lunch & Learn: AI Frontiers; Palette & Palate: Art Meets Cooking Experience	$45
2025-06-12	Thunderstorm, 28 C	Soundtrack Picnic: Lunchtime Movies & Melodies (indoor rain backup); Tech & Film Fusion Night	$30

Table 4. The actual final itinerary (TravelPlan v2) from the successful run, identical to the draft TravelPlan v1. Total cost: $130 of a $130 budget.

9. Value Statement

The value of this project extends well beyond producing a single working trip planner. The pattern demonstrated here that a lightweight generator agent paired with an independent, code-based evaluation harness and a tool-using repair agent, is directly transferable to any domain where a language model needs to produce a structured artifact that must satisfy multiple, independently checkable constraints simultaneously. Expense-report generation, meeting-scheduling assistants, procurement or vendor-selection recommendations, and simple software configuration generation all share the same shape as the trip-planning problem: a set of hard constraints, such as budget, availability, and policy compliance, a set of soft preferences, traveler

interests in this case, and a requirement that every referenced entity, whether that is an activity, a vendor, or a configuration flag, be real rather than invented.

Grounding the system in Pydantic models at every boundary delivers value that is easy to overlook because it is largely defensive: it converts the vague, high-surface-area failure mode of the model said something wrong into a small number of precisely typed, immediately actionable validation errors, each pointing at exactly which field violated which constraint. This is the same discipline that underlies FastAPI, much of the modern Python API ecosystem, and, notably, the validation layer inside the OpenAI Python SDK itself, so adopting it in an agentic context is not a novel invention so much as an extension of an already well-proven engineering practice into a new setting where the producer of the data is a language model rather than an HTTP client.

The ReAct-based revision agent demonstrates a second, distinct source of value: the ability to make small, targeted, auditable repairs to an artifact rather than regenerating it wholesale on every iteration. Every THOUGHT, ACTION, and OBSERVATION in the loop is retained in the conversation history, which means a developer debugging a failed run does not have to guess why the model made a change; the full reasoning trace is available for inspection, a property that matters enormously in any regulated or safety-conscious deployment where an agent's decisions may need to be explained or audited after the fact.

Finally, the project has direct pedagogical and organizational value as a template. Because the four tools, the evaluation harness, and the two-agent structure are all decoupled from the specific domain of trip planning, most of this codebase, the ChatAgent base class, the ReAct loop implementation, the evaluation-and-tool pattern, and the prompt-design conventions, can be lifted with only the domain-specific system prompts, Pydantic models, and evaluation functions swapped out, turning a several-week engineering exercise into a reusable scaffold for the next agentic system a team needs to build.

10. Limitations

The most significant limitation of the current system is the stochastic, imperfect instruction-following of the underlying language model, particularly inside the ReAct revision loop. Even with a heavily hardened system prompt that explicitly forbids calling final_answer_tool while any evaluation failure remains outstanding, GPT-4.1 was observed on multiple real runs to finalize an itinerary prematurely, either with an inexact total_cost or with an unaddressed per-day activity shortfall. The mitigations implemented in this project, explicit per-day checklists, single-source-of-truth cost computation via a calculator tool, and repeated framing of the gating rule,

measurably reduce the frequency of this failure but do not eliminate it, and the honest position is that the evaluation harness, not the prompt, is what ultimately protects the system's correctness guarantee.

The mocked data layer is a second material limitation. Both the weather and activities APIs return synthetic, pre-authored data scoped to a single fictional destination and a narrow six-day date range, and the destination-name binding between the notebook's VacationInfo and the underlying mocked data proved to be an actual, reproducible bug during this project, a naming mismatch between two destination names used in different derivatives of the starter files, which silently produced empty weather and activity results rather than a loud, immediate error. A production system would need to replace this mock layer with real, rate-limited, occasionally-unavailable third-party APIs, which introduces failure modes, such as partial data, API timeouts, and inconsistent schemas across providers, that this project's controlled environment does not exercise at all.

The evaluation harness itself, while a genuine strength of the architecture, is not exhaustive. It cannot check subjective itinerary quality, such as whether the ordering of activities within a day is sensible, whether travel time between two activities' locations is realistic, or whether an itinerary is merely constraint-satisfying rather than genuinely enjoyable, since none of these properties were encoded as explicit evaluation functions. The single evaluation function that does rely on an LLM call rather than deterministic code, the weather-compatibility check, is itself subject to the same reliability concerns as any other language-model judgment, and a sufficiently ambiguous activity description could plausibly receive different verdicts on different runs.

Finally, cost and latency scale with the number of ReAct loop iterations the revision agent requires, and a pipeline that occasionally needs eight or more tool-calling turns to converge is meaningfully more expensive and slower than the single-pass ItineraryAgent, a tradeoff that is appropriate for this project's correctness-first design goal but that would need explicit monitoring, and likely a hard cost ceiling per revision request, in any production deployment serving real users at scale.

11. Conclusion

This project set out to answer a narrow but consequential question: can a language model be trusted to plan something as constraint-heavy as a multi-day, multi-traveler trip on its own, or does it need external scaffolding to be reliable? The evidence gathered across this project points firmly toward the second answer. A single, well-crafted chain-of-thought prompt produced a

first-draft itinerary that was frequently, but not reliably, fully compliant with every stated constraint, and a naive framing of an iterative repair loop, even with a strong model like GPT-4.1 and an unambiguous tool interface, was not sufficient on its own to guarantee that the model would recognize and act on every remaining violation before declaring itself finished.

What did prove reliable, consistently, across every run examined during this project, was the code-based evaluation harness: six functions that never hallucinate, never get talked out of a failing result, and never lose track of which day of the trip they are checking. Anchoring both agents to this harness, rather than to each other's or their own stated confidence, is the architectural decision that ultimately made the system trustworthy, and the iterative process of using real failure traces from that harness to progressively harden the revision agent's system prompt is a concrete demonstration of the empirical, evaluation-driven development loop that responsible agentic AI engineering increasingly requires.

The two-agent, four-tool, six-function architecture documented in this report is intentionally modest in scope, a three-day trip for two travelers with a handful of mocked activities, but the underlying pattern scales conceptually to considerably more complex planning problems, provided the same discipline is maintained: every constraint that matters must be encoded as an independently checkable function, every structured artifact must be validated against an explicit schema, and every autonomous repair loop must be required to prove its own correctness against that same harness before it is allowed to stop.

The system as built is not production-ready, and Section 10 documents honestly why: stochastic non-convergence, a mocked and narrow data layer, an incomplete notion of itinerary quality, and unmanaged cost and latency at the tail of the loop-iteration distribution. But as a working demonstration of chain-of-thought generation, structured Pydantic validation, ReAct-style tool use, and evaluation-driven iterative prompt engineering operating together on a single, coherent problem, it succeeds at its stated purpose, and it leaves behind a template that is substantially more valuable than the trip planner itself.

Taken together, the executive summary, the architecture, the tool and prompt design sections, and the results presented in this report tell a single, consistent story: reliability in agentic AI is not purchased with a cleverer prompt alone, it is purchased with the combination of a clear prompt, a typed data contract, a narrow and well-documented toolset, and an evaluation harness that is willing to say no. That combination, more than any single technique in isolation, is the lasting contribution of this project.

12. References

All references below were verified to be live, accessible sources at the time this report was prepared.

[1] Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., & Cao, Y. (2023). ReAct: Synergizing reasoning and acting in language models. arXiv:2210.03629. https://arxiv.org/abs/2210.03629

[2] Wei, J., Wang, X., Schuurmans, D., Bosma, M., Ichter, B., Xia, F., Chi, E., Le, Q., & Zhou, D. (2022). Chain-of-thought prompting elicits reasoning in large language models. arXiv:2201.11903. https://arxiv.org/abs/2201.11903

[3] OpenAI. (2025). Introducing GPT-4.1 in the API. https://openai.com/index/gpt-4-1/

[4] OpenAI. GPT-4.1 prompting guide. OpenAI Cookbook. https://github.com/openai/openai-cookbook/blob/main/examples/gpt4-1_prompting_guide.ipynb

[5] OpenAI. Function calling. OpenAI API documentation. https://developers.openai.com/api/docs/guides/function-calling

[6] Pydantic. Pydantic validation documentation (v2). https://docs.pydantic.dev/latest/